

# CLOUD-99: Best Practice Summary

Series: CLOUD | Notebook: 99 | Created: March 2026 | Last Updated: 03/26/2026

## Overview

This notebook distills every actionable best practice from the CLOUD series (CLOUD-01 through CLOUD-08) into definitive, priority-ranked tables. Each entry specifies the exact setting, value, or action to take. No hedging — these are the recommended configurations.

## Table of Contents

- 1. [Connection & Authentication](#)
- 2. [AWS Integration](#)
- 3. [Azure Integration](#)
- 4. [GCP Integration](#)
- 5. [Kubernetes \(EKS / AKS / GKE\)](#)
- 6. [Serverless Monitoring](#)
- 7. [Log Ingestion](#)
- 8. [Cost Optimization](#)
- 9. [Multi-Cloud Governance](#)
- 10. [Alerting & Observability](#)

## 1. Connection & Authentication

| # | Best Practice                                       | Recommended Setting/Value                                                       | Priority | Source   |
|---|-----------------------------------------------------|---------------------------------------------------------------------------------|----------|----------|
| 1 | Use Clouds app direct connections for SaaS          | <b>Clouds app &gt; Add connection</b> — no ActiveGate required                  | Critical | CLOUD-01 |
| 2 | Reserve classic ActiveGate polling for Managed only | ActiveGate-based polling for Dynatrace Managed or strict network isolation only | Critical | CLOUD-01 |

| #  | Best Practice                                                 | Recommended Setting/Value                                                                                | Priority    | Source   |
|----|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-------------|----------|
| 3  | AWS: Use IAM role-based auth (STS AssumeRole)                 | IAM Role with trust policy to Dynatrace AWS account; never use long-lived access keys in production      | Critical    | CLOUD-02 |
| 4  | AWS: Attach ReadOnlyAccess or custom least-privilege policy   | Never grant AdministratorAccess to the monitoring role                                                   | Critical    | CLOUD-02 |
| 5  | Azure: Use Azure Native Dynatrace Service                     | Deploy from Azure Marketplace for zero-infrastructure setup with unified billing                         | Recommended | CLOUD-05 |
| 6  | Azure: Use managed identity when ActiveGate runs on Azure VM  | Eliminates credential rotation entirely                                                                  | Critical    | CLOUD-05 |
| 7  | Azure: Assign Reader role at subscription scope               | Scope: subscription or management group; role: Reader or Monitoring Reader                               | Critical    | CLOUD-05 |
| 8  | Azure: Use certificate-based auth for production (non-native) | Client secret for dev only; certificates for production Entra ID app registrations                       | Recommended | CLOUD-05 |
| 9  | GCP: Create dedicated service account with minimal roles      | Assign roles/monitoring.viewer , roles/compute.viewer , roles/container.viewer , roles/cloudasset.viewer | Critical    | CLOUD-06 |
| 10 | GCP: Use dedicated project for monitoring resources           | Isolate Dynatrace service accounts, Pub/Sub topics, and billing in a separate GCP project                | Recommended | CLOUD-06 |
| 11 | GCP: Use Helm on GKE for push-based integration               | Deploy via Pub/Sub for metrics and logs; no ActiveGate needed for SaaS                                   | Recommended | CLOUD-06 |

## 2. AWS Integration

| #  | Best Practice                                          | Recommended Setting/Value                                                                                                                                                                              | Priority    | Source   |
|----|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| 12 | Enable services in tiers, not all at once              | <b>Tier 1 (always):</b> EC2, ELB, RDS, Lambda.<br><b>Tier 2:</b> S3, SQS, SNS, DynamoDB, ECS/EKS. <b>Tier 3:</b> CloudFront, Route 53, Kinesis, Step Functions. <b>Tier 4:</b> SageMaker, Bedrock, IoT | Critical    | CLOUD-02 |
| 13 | Use tag-based resource filtering                       | Tag key: <code>dynatrace-monitored</code> , value: <code>true</code> ; only monitored resources are discovered                                                                                         | Critical    | CLOUD-02 |
| 14 | Scope monitoring to active regions only                | Disable regions with no workloads in the Clouds app connection settings                                                                                                                                | Recommended | CLOUD-02 |
| 15 | Account for CloudWatch metric delays in alert windows  | Standard metrics: 5–10 min delay. Set alert evaluation window $\geq$ 15 min for cloud metrics                                                                                                          | Critical    | CLOUD-02 |
| 16 | Enable CloudWatch detailed monitoring for critical EC2 | Detailed monitoring: 1-min granularity (additional cost); standard: 5-min                                                                                                                              | Recommended | CLOUD-02 |
| 17 | Combine cloud integration with OneAgent on compute     | Cloud integration = cloud context (tags, AZ, instance type); OneAgent = deep observability (traces, code-level). Use both                                                                              | Critical    | CLOUD-01 |
| 18 | Review CloudWatch billing dashboard monthly            | Correlate API cost spikes with Dynatrace service additions                                                                                                                                             | Recommended | CLOUD-02 |

## 3. Azure Integration

| #  | Best Practice                                   | Recommended Setting/Value                                                                                                                | Priority    | Source   |
|----|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| 19 | Map Azure resource groups to Dynatrace Segments | One Segment per resource group or application for access control and scoped views                                                        | Critical    | CLOUD-05 |
| 20 | Propagate Azure tags to Dynatrace               | Tags must flow through automatically; verify <code>environment</code> , <code>team</code> , <code>cost-center</code> appear in Dynatrace | Recommended | CLOUD-05 |

| #  | Best Practice                                           | Recommended Setting/Value                                                                                                  | Priority    | Source   |
|----|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| 21 | Use naming convention for resource groups               | Pattern: <code>rg-&lt;application&gt;-&lt;environment&gt;-&lt;region&gt;</code> (e.g., <code>rg-myapp-prod-eastus</code> ) | Recommended | CLOUD-05 |
| 22 | Forward Azure logs via Event Hub or Diagnostic Settings | Use Azure Diagnostic Settings to stream to Dynatrace; Event Hub for high-volume                                            | Recommended | CLOUD-07 |

## 4. GCP Integration

| #  | Best Practice                                        | Recommended Setting/Value                                                                                                             | Priority    | Source   |
|----|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| 23 | Use Workload Identity for GKE pods                   | Replace service account JSON keys with Workload Identity for DynaKube and integration pods                                            | Critical    | CLOUD-06 |
| 24 | Tag all GCP resources with standard labels           | Required labels: <code>env</code> , <code>team</code> , <code>service</code> , <code>cost-center</code> ; propagate as Dynatrace tags | Recommended | CLOUD-06 |
| 25 | Propagate GCP project ID as a Dynatrace tag          | Tag key: <code>gcp.project</code> ; enables cross-project filtering and Segment creation                                              | Recommended | CLOUD-06 |
| 26 | Create Dynatrace Segments per GCP project            | One Segment per project for access control, cost attribution, and scoped dashboards                                                   | Recommended | CLOUD-06 |
| 27 | Monitor Pub/Sub message volume for cost control      | Filter logs at Cloud Logging export to reduce Pub/Sub throughput and costs                                                            | Recommended | CLOUD-06 |
| 28 | Use Cloud Monitoring metrics scope for multi-project | Aggregate metrics from multiple projects into a single metrics scope rather than separate integrations                                | Optional    | CLOUD-06 |

## 5. Kubernetes (EKS / AKS / GKE)

| #  | Best Practice                                                        | Recommended Setting/Value                                                                          | Priority | Source   |
|----|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|----------|----------|
| 29 | Deploy DynaKube in CloudNativeFullStack mode for standard node pools | <code>spec.oneAgent.cloudNativeFullStack</code> — full agent injection + infrastructure monitoring | Critical | CLOUD-03 |

| #  | Best Practice                                                               | Recommended Setting/Value                                                                                                    | Priority    | Source             |
|----|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------|--------------------|
| 30 | Use ApplicationMonitoring mode for Fargate and GKE Autopilot                | <code>spec.oneAgent.applicationMonitoring</code> with <code>useCSIDriver: false</code> for Fargate                           | Critical    | CLOUD-03, CLOUD-06 |
| 31 | Add tolerations for tainted node groups                                     | <code>tolerations: [{effect: NoSchedule, key: node-role, operator: Exists}]</code> to ensure OneAgent runs on all nodes      | Critical    | CLOUD-03           |
| 32 | Enable kubernetes-monitoring capability on ActiveGate                       | <code>spec.activeGate.capabilities: [kubernetes-monitoring, routing]</code>                                                  | Critical    | CLOUD-03           |
| 33 | Use nodeSelector to exclude Windows nodes                                   | <code>nodeSelector: {kubernetes.io/os: linux}</code> in DynaKube spec                                                        | Recommended | CLOUD-03           |
| 34 | Enforce namespace labels for cost allocation                                | Require <code>team</code> , <code>cost-center</code> , <code>environment</code> labels on all namespaces                     | Recommended | CLOUD-03           |
| 35 | Set resource requests and limits on all pods                                | Accurate requests enable fair cost attribution and prevent resource contention                                               | Recommended | CLOUD-03           |
| 36 | Use Dynatrace for workload monitoring, Container Insights for control plane | Keep CloudWatch Container Insights / Azure Monitor for control plane logs; Dynatrace for distributed tracing and AI alerting | Recommended | CLOUD-03           |
| 37 | Use IRSA (AWS) or Workload Identity (GCP) for pod IAM                       | Never mount service account JSON keys into pods; use cloud-native identity federation                                        | Critical    | CLOUD-03, CLOUD-06 |

## 6. Serverless Monitoring

| #  | Best Practice                                          | Recommended Setting/Value                                                     | Priority | Source   |
|----|--------------------------------------------------------|-------------------------------------------------------------------------------|----------|----------|
| 38 | Install Dynatrace Lambda Layer on all Lambda functions | Enables distributed tracing, code-level visibility, and direct log collection | Critical | CLOUD-04 |

| #  | Best Practice                                        | Recommended Setting/Value                                                                                                | Priority    | Source   |
|----|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| 39 | Use meaningful Lambda function names                 | Avoid auto-generated names; use <code>&lt;service&gt;-&lt;function&gt;-&lt;env&gt;</code> pattern for identification     | Recommended | CLOUD-04 |
| 40 | Monitor concurrency headroom                         | Alert when <code>cloud.aws.lambda.concurrentExecutions</code> exceeds 80% of account limit (default: 1,000/region)       | Critical    | CLOUD-04 |
| 41 | Track cold start ratio via spans                     | Query <code>faas.coldstart == true</code> from spans; high ratio indicates need for provisioned concurrency or SnapStart | Recommended | CLOUD-04 |
| 42 | Use error-to-invocation ratio, not raw error count   | <code>error_pct = errors / invocations * 100</code> ; alert threshold: > 1% for critical functions                       | Critical    | CLOUD-04 |
| 43 | Correlate API Gateway latency with Lambda duration   | End-to-end latency = API GW overhead + Lambda duration; monitor both                                                     | Recommended | CLOUD-04 |
| 44 | Monitor DynamoDB call duration from Lambda spans     | Filter: <code>span.kind == "client"</code> , <code>db.system == "dynamodb"</code> ; alert on p95 > baseline              | Recommended | CLOUD-04 |
| 45 | Set alert evaluation window longer than metric delay | Cloud metric delay: 5–10 min. Minimum evaluation window for Lambda alerts: 15 min                                        | Critical    | CLOUD-02 |

## 7. Log Ingestion

| #  | Best Practice                                          | Recommended Setting/Value                                                                        | Priority | Source   |
|----|--------------------------------------------------------|--------------------------------------------------------------------------------------------------|----------|----------|
| 46 | Use Amazon Data Firehose for CloudWatch log forwarding | Fully managed, auto-scaling, no custom code. Buffer: 1 MB or 60 seconds. Enable GZIP compression | Critical | CLOUD-07 |

| #  | Best Practice                                                        | Recommended Setting/Value                                                                                                                                                | Priority    | Source   |
|----|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| 47 | Migrate legacy Lambda log forwarder to Firehose                      | <code>dynatrace-aws-log-forwarder</code> is <b>deprecated</b> — do not use for new deployments                                                                           | Critical    | CLOUD-07 |
| 48 | Use Lambda Layer log collection for Lambda functions                 | When Lambda Layer is deployed for tracing, use its built-in log collection instead of Firehose for Lambda logs                                                           | Recommended | CLOUD-07 |
| 49 | Apply subscription filter patterns at CloudWatch level               | Filter pattern examples: <code>?"ERROR"</code><br><code>?"WARN"</code> <code>?"CRITICAL"</code> for Lambda;<br><code>""</code> (all) for application services            | Critical    | CLOUD-07 |
| 50 | Drop debug/trace logs at source                                      | Pre-filter: exclude DEBUG and TRACE in CloudWatch subscription filters.<br>Estimated savings: 40–60%                                                                     | Critical    | CLOUD-07 |
| 51 | Filter health check log lines at source                              | Remove <code>/health</code> , <code>/ready</code> , <code>/live</code> endpoint noise before forwarding.<br>Savings: 10–30%                                              | Recommended | CLOUD-07 |
| 52 | Use OpenPipeline for enrichment, routing, and fine-grained filtering | Add <code>cloud.provider</code> , route by log group to specific Grail buckets, drop remaining noise                                                                     | Recommended | CLOUD-07 |
| 53 | Route logs to purpose-specific Grail buckets                         | Route <code>/aws/lambda/*</code> → <code>lambda_logs</code> , <code>/ecs/*</code> → <code>application_logs</code> , <code>/aws/rds/*</code> → <code>database_logs</code> | Recommended | CLOUD-07 |
| 54 | Enable S3 backup on Firehose delivery stream                         | Backup failed or all records to S3 for disaster recovery and compliance                                                                                                  | Recommended | CLOUD-07 |
| 55 | Azure: Use Event Hub or Diagnostic Settings for log forwarding       | Stream activity and resource logs directly to Dynatrace                                                                                                                  | Recommended | CLOUD-07 |
| 56 | GCP: Use Pub/Sub via GKE integration for log forwarding              | Push-based delivery; filter at Cloud Logging export level to reduce volume                                                                                               | Recommended | CLOUD-07 |

## 8. Cost Optimization

| #  | Best Practice                                   | Recommended Setting/Value                                                                                                     | Priority    | Source             |
|----|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------|--------------------|
| 57 | Disable unused cloud service monitoring         | Only enable AWS/Azure/GCP services your applications depend on; disable the rest                                              | Critical    | CLOUD-02           |
| 58 | Tag-based filtering is the #1 cost lever        | Monitor only resources tagged <code>dynatrace-monitored: true</code> ; reduces both DPS and cloud API costs                   | Critical    | CLOUD-02           |
| 59 | Disable individual metrics you do not need      | Per-service metric selection in Clouds app or Settings UI                                                                     | Recommended | CLOUD-02           |
| 60 | Consider CloudWatch Metric Streams at scale     | Streaming can be cheaper than polling for >100 resources with frequent metric retrieval                                       | Optional    | CLOUD-02           |
| 61 | Set bucket-level retention policies in Grail    | Low-value logs: 7–14 days. Standard logs: 35 days. Compliance logs: 365+ days                                                 | Critical    | CLOUD-07           |
| 62 | Monitor log volume weekly                       | Query: <code>fetch logs, from:-7d \   makeTimeseries log_count = count(), interval:1h</code> — investigate spikes immediately | Recommended | CLOUD-07           |
| 63 | Rightsize overprovisioned instances             | Alert threshold: CPU < 10% sustained over 24h. Action: downsize or terminate                                                  | Recommended | CLOUD-08           |
| 64 | Identify zero-traffic services for decommission | Query services with no spans/requests over 7 days; flag as decommission candidates                                            | Optional    | CLOUD-08           |
| 65 | Enforce K8s resource requests/limits            | Pods without requests/limits cause uncontrolled resource usage and inaccurate cost allocation                                 | Recommended | CLOUD-03, CLOUD-08 |

## 9. Multi-Cloud Governance

| #  | Best Practice                                              | Recommended Setting/Value                                                                                                                                             | Priority    | Source   |
|----|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| 66 | Apply consistent tag schema across all providers           | Required tags: <code>cloud-provider</code> , <code>environment</code> , <code>team</code> , <code>application</code> , <code>cost-center</code> , <code>region</code> | Critical    | CLOUD-08 |
| 67 | Normalize region names across providers                    | Use normalized values: <code>us-east</code> , <code>eu-west</code> , <code>ap-south</code> — not provider-specific names                                              | Recommended | CLOUD-08 |
| 68 | Create Dynatrace Segments per cloud provider + environment | One Segment per combination (e.g., <code>AWS-Production</code> , <code>Azure-Staging</code> ) for access control                                                      | Critical    | CLOUD-08 |
| 69 | Enforce tagging compliance with automation                 | Reject or alert on untagged cloud resources; query: <code>fetch dt.entity.host \\\ filter isNull(tags)</code>                                                         | Critical    | CLOUD-08 |
| 70 | Define SLOs at the service level, not infrastructure level | SLOs are cloud-agnostic: measure user experience (latency, error rate, availability)                                                                                  | Recommended | CLOUD-08 |
| 71 | Route alerts by team, not by cloud provider                | Dynatrace Workflows route to the application owner regardless of which cloud has the issue                                                                            | Critical    | CLOUD-08 |
| 72 | Use unified runbooks referencing DQL queries               | Runbooks should use Dynatrace DQL, not provider-specific CLI tools                                                                                                    | Recommended | CLOUD-08 |
| 73 | Conduct monthly utilization reviews across all providers   | Review underutilized hosts (CPU < 10%), untagged resources, and zero-traffic services                                                                                 | Recommended | CLOUD-08 |

## 10. Alerting & Observability

| #  | Best Practice                                            | Recommended Setting/Value                                                                         | Priority | Source   |
|----|----------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------|----------|
| 74 | Use Davis AI for automatic cross-cloud anomaly detection | Enable Davis anomaly detection on all monitored entities; do not rely solely on static thresholds | Critical | CLOUD-08 |

| #  | Best Practice                                                | Recommended Setting/Value                                                                                                                   | Priority    | Source             |
|----|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-------------|--------------------|
| 75 | Set alert evaluation windows $\geq 15$ min for cloud metrics | Cloud metric delay: 5–15 min. A 1-min evaluation window on 5-min metrics produces false negatives                                           | Critical    | CLOUD-02           |
| 76 | Use OneAgent metrics for sub-minute alerting                 | For alerts requiring $< 5$ min evaluation, use <code>dt.host.*</code> or <code>dt.process.*</code> metrics from OneAgent, not cloud metrics | Critical    | CLOUD-02           |
| 77 | Combine cloud integration with OneAgent on all compute       | Cloud integration: infrastructure context. OneAgent: processes, traces, code-level diagnostics. Both required for full-stack visibility     | Critical    | CLOUD-01           |
| 78 | Build unified health dashboards showing all providers        | Components: host CPU (all providers), active Davis problems, service error rates, log error trends, K8s container metrics                   | Recommended | CLOUD-08           |
| 79 | Deduplicate alerts using Davis correlation                   | Davis automatically correlates related issues across providers; do not create redundant static alerts                                       | Recommended | CLOUD-08           |
| 80 | Forward control plane logs to Dynatrace for unified analysis | EKS: Container Insights logs. AKS: Azure Monitor logs. GKE: Cloud Logging. Forward all to Grail for cross-platform analysis                 | Recommended | CLOUD-03, CLOUD-06 |

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*